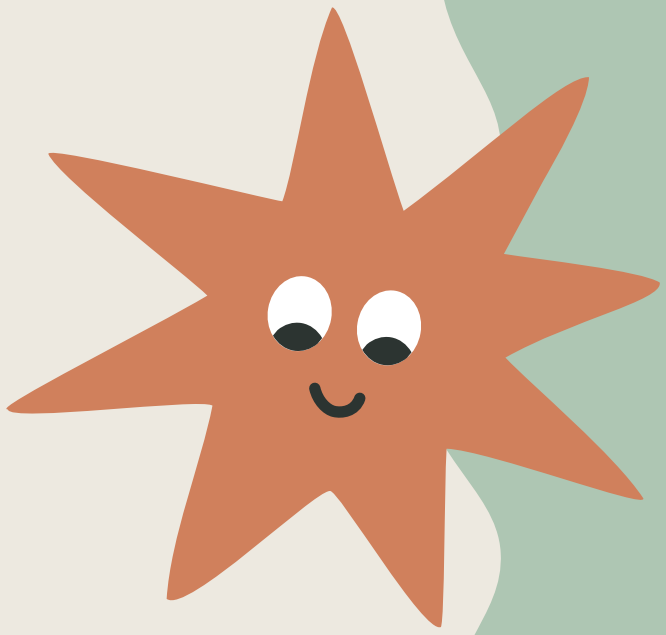
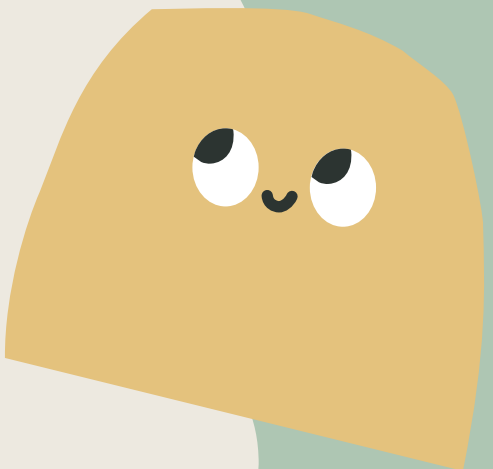
Tuples & Sets in Python

Lecture 8





Learning Objectives

➢By the end of this lecture, students should be able to:

- Understand what tuples are and why they are used
- Create and access tuples
- Use tuple indexing and slicing
- Understand tuple immutability
- Understand what sets are
- Create and modify sets
- Perform union, intersection, difference, and symmetric difference
- Understand when to use **lists, tuples, and sets**
- Apply tuples and sets to practical programming and AI/data-processing examples

Prepared By: Ehtisham Rasheed

What is a Tuple?

- A **tuple** is an ordered collection of elements that cannot be changed after creation.
- Example:

```
student = ("Ali", 20, 3.45)
```

```
numbers = (10, 20, 30, 40)
```

```
students = ("Ali", "Ahmed", "Sara")
```

```
empty = ()
```

Prepared by: Ehtisham Rasheed

Tuple with One Element

- There is an important special case.
- This: `number = (10)` is **not a tuple**.
- Python considers it an integer. Output

```
number = (10)

print(type(number))
```

```
<class 'int'>
```

- To create a single-element tuple, we need a comma:

```
number = (10,)

print(type(number))
```

```
<class 'tuple'>
```

Prepared By: Ehtisham Rasheed

Accessing Tuple Elements

- Tuples support indexing just like lists.

```
students = ("Ali", "Ahmed", "Sara", "Ayesha")

print(students[0])
print(students[1])
print(students[2])
```

- Python uses zero-based indexing:

Element:	Ali	Ahmed	Sara	Ayesha
Index:	0	1	2	3

- Tuples also support negative indexing.

```
students = ("Ali", "Ahmed", "Sara", "Ayesha")

print(students[-1])
print(students[-2])
```

Prepared By Engr. Rasheed

Tuple Slicing

➢Tuples also support slicing.

```
numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])
```

Output

```
(20, 30, 40)
```

➢We can also use:

```
print(numbers[:3])
print(numbers[2:])
print(numbers[::-1])
```

Prepared By: Ehtisha Rasheed

Tuples Are Immutable

- This is the most important property of tuples.
- **Immutable** means
  - Once a tuple is created, its elements cannot be changed.
- Consider:

```
students = ("Ali", "Ahmed", "Sara")
```

- We can't do:

```
students[1] = "Usman"
```

- Python will generate an error.

Prepared By: Ehtisham Rasheed

Why Do We Need Tuples?

- Suppose we have the coordinates of a location:

```
location = (32.49, 74.52)
```

- We probably don't want those coordinates accidentally changed.
- A tuple is appropriate because the data represents a fixed collection.
- Other examples:

```
rgb = (255, 128, 0)
```

```
date = (2026, 10, 3)
```

```
student_id = (2026, 105, "CS")
```

```
dimensions = (1920, 1080)
```

Prepared By: Ehtisham Rasheed

Tuple Methods

- Tuples have fewer methods than lists because they cannot be modified.
- Two particularly useful methods are:
- count()
  - count() tells us how many times a value appears.

```
numbers = (10, 20, 10, 30, 10)

print(numbers.count(10))
```

- index()
  - index() returns the position of the first occurrence.

```
students = ("Ali", "Ahmed", "Sara", "Ahmed")

print(students.index("Ahmed"))
```

Prepared By Mohammad Ihtisham Rasheed

Looping Through Tuples

- We can use a for loop with tuples.

```
students = ("Ali", "Ahmed", "Sara", "Ayesha")

for student in students:
    print(student)
```

- Output:

```
Ali
Ahmed
Sara
Ayesha
```

Prepared By: Ehtisham Rasheed

Tuple Unpacking

- One of the most useful features of tuples is **unpacking**.
- Consider:

```
student = ("Ali", 20, 3.45)
```

- We can write:

```
name, age, gpa = student
```

- Now:

```
print(name)
print(age)
print(gpa)
```

Output

```
Ali
20
3.45
```

Prepared By: Dr. Tisham Rasheed

What is a Set?

- A set is an **unordered collection of unique elements**.
- Example:

```
numbers = {10, 20, 30, 40}
```

- Sets use curly braces:

```
numbers = {10, 20, 30, 40}


print(numbers)
```

- Unlike lists and tuples, sets do not maintain duplicate values
- The duplicate values are automatically removed
- The order displayed may not be the same as the order in which the elements were written.

Prepared By: Ebraham Rasheed

Creating an Empty Set

- Be careful here.
- This `empty = {}` does **not** create an empty set.
- It creates an empty dictionary
- To create an empty set: `empty = set()`

- Example:

```
my_set = set()

print(type(my_set))
```

Output

```
<class 'set'>
```

Prepared By: Ehtesham Rasheed

Set Membership

- Sets are very efficient for checking whether an element exists

```
students = {"Ali", "Ahmed", "Sara"}

print("Ali" in students)
print("Usman" in students)
```

- Output:

```
True
False
```

Prepared By: Engr. Usham Rasheed

Adding Elements to a Set

```
students = {"Ali", "Ahmed"}

students.add("Sara")

print(students)
```

Adding Multiple Elements

```
students = {"Ali", "Ahmed"}

students.update(["Sara", "Ayesha"])

print(students)
```

Ehtisham Rasheed

Removing Elements — remove() & discard()

```
students = {"Ali", "Ahmed", "Sara"}

students.remove("Ahmed")

print(students)
```

```
students = {"Ali", "Ahmed", "Sara"}

students.discard("Usman")

print(students)
```

Prepared By: Ehtesham Sheed

➢ Difference

- remove() → Error if element doesn't exist
- discard() → No error

Clearing a Set

➢Use:

```
clear()
```

➢Example:

```
students = {"Ali", "Ahmed", "Sara"}

students.clear()

print(students)
```

➢Output:

```
set()
```

Prepared by Hisham Rasheed

Set Operations

- Union combines elements from both sets.

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A | B)
```

```
A.union(B)
```

```
{1, 2, 3, 4, 5, 6}
```

- Intersection finds elements that exist in **both** sets.

```
print(A & B)
```

```
A.intersection(B)
```

```
{3, 4}
```

- Difference finds elements that exist in the first set but not the second.

```
print(A - B)
```

```
{1, 2}
```

- Symmetric difference gives elements that are in **either set but not in both**.

```
print(A ^ B)
```

```
{1, 2, 5, 6}
```

Prepared By: Eutisham Rasheed

Removing Duplicates Using a Set

- One of the most practical uses of sets is removing duplicates from a list.
- Suppose:

```
numbers = [10, 20, 10, 30, 20, 40, 30]
```

- We can write:

```
unique_numbers = set(numbers)

print(unique_numbers)
```

- Result:

```
{10, 20, 30, 40}
```

- This is much easier than manually checking every duplicate

Prepared By Nisham Rasheed

Looping Through a Set

- We can loop through a set:

```
skills = {"Python", "Java", "C++", "Flutter"}

for skill in skills:
    print(skill)
```

- However, remember Sets are unordered collections.
- Therefore, don't build a program that depends on a particular set iteration order.

Prepared By: Ehti... Rasheed

List vs Tuple vs Set

Feature	List	Tuple	Set
Syntax	`[]`	`()`	`{}`
Ordered	Yes	Yes	No positional order
Indexed	Yes	Yes	No
Mutable	Yes	No	Yes
Duplicates	Allowed	Allowed	Not allowed
Slicing	Yes	Yes	No
`append()`	Yes	No	No
Set operations	No	No	Yes
Best for	Changeable collection	Fixed collection	Unique values

When Should We Use a List?

➢Use a **list** when:

- We have multiple values
- Order matters
- Data may change
- Duplicates are acceptable
- You need indexing and slicing

Prepared By: Ehtisham Rasheed

When Should We Use a Tuple?

➢Use a **tuple** when:

- Data should not change
- Order matters
- You want indexing
- The values represent a fixed group

Prepared By: Ehtisham Rasheed

When Should We Use a Set?

➢Use a **set** when:

- You need unique values
- You frequently check membership
- You need union/intersection/difference
- Order is not important

Prepared By: Ehtisham Rasheed